

GOOGLE SUMMER OF CODE 2026

C2SI - Ceylon Computer Science Institute

Per-Session GDB Sandbox Architecture with Real-Time Debugging for GDB-UI

Field	Details
Applicant	Udaybir Singh
Email	writetoudaybir@gmail.com
GitHub	github.com/Uday9909
University	MIT Art, Design & Technology University, Pune
Year	2nd Year B.Tech Computer Science
Timezone	IST (UTC+5:30)
Availability	35-40 hours/week (May through August)
Project Length	350 Hours - Medium Difficulty
Mentors	Shubh Mehta, Nipuna Weerasekara



Google Summer of Code

1. Abstract

GDB-UI currently supports debugging for only a single user because the backend shares one global GDB controller across all requests. This proposal implements a per-session sandbox architecture that isolates each user's debugging environment, enables real-time GDB output streaming via WebSockets, and completes the Docker-based project sandboxing needed for full multi-user isolation. Building directly on the foundation established in GSoC 2024, this work addresses the next major architectural challenge: making GDB-UI a truly multi-user, production-ready debugging platform.

The proposal is structured in three phases. Phase 1 replaces the global `gdb_controller` with a `SessionManager` that maps each user's `session_id` to their own isolated GDB instance. Phase 2 introduces WebSocket streaming so output reaches the React frontend in real time, including a per-session reader thread architecture to handle `pygdbmi`'s blocking I/O model correctly. Phase 3 moves each project into an isolated Docker container - including the non-trivial work of GDB process attachment inside a container and `stdin/stdout` piping through docker exec.

Already shipped:

A working implementation of Phase 1 is open as PR #131 with 15 passing tests, security fixes, per-session filesystem isolation, and a live demo page. The architecture described here has been validated against real GDB processes - this proposal describes what I found and what still needs to be built.

2. Problem Statement

The current backend uses two global variables shared across every HTTP request:

```
gdb_controller = None
program_name    = None
```

This creates three critical limitations that block the project's roadmap goals.

2.1 Single-User Only

If User A is debugging `program1` and User B starts debugging `program2`, the server calls `start_gdb_session()` which silently kills User A's process and starts a new one. No error. No warning. User A loses their entire debug session.

2.2 No Real-Time Output

The frontend sends HTTP requests and waits for complete responses. Long operations like stepping through loops block the entire UI with no live feedback. There is also a deeper problem: `pygdbmi`'s `controller.write()` is a blocking call. Any real-time streaming architecture must account for this explicitly. A naive flask-socketio integration without a dedicated reader thread per session will block the event loop, producing the same frozen-UI experience.

2.3 No Filesystem Isolation

All compiled binaries land in a single shared `output/` directory. If two users compile a file named `program.cpp`, the second binary overwrites the first at the filesystem level before the GDB layer is even reached.

2.4 Architecture: Before and After

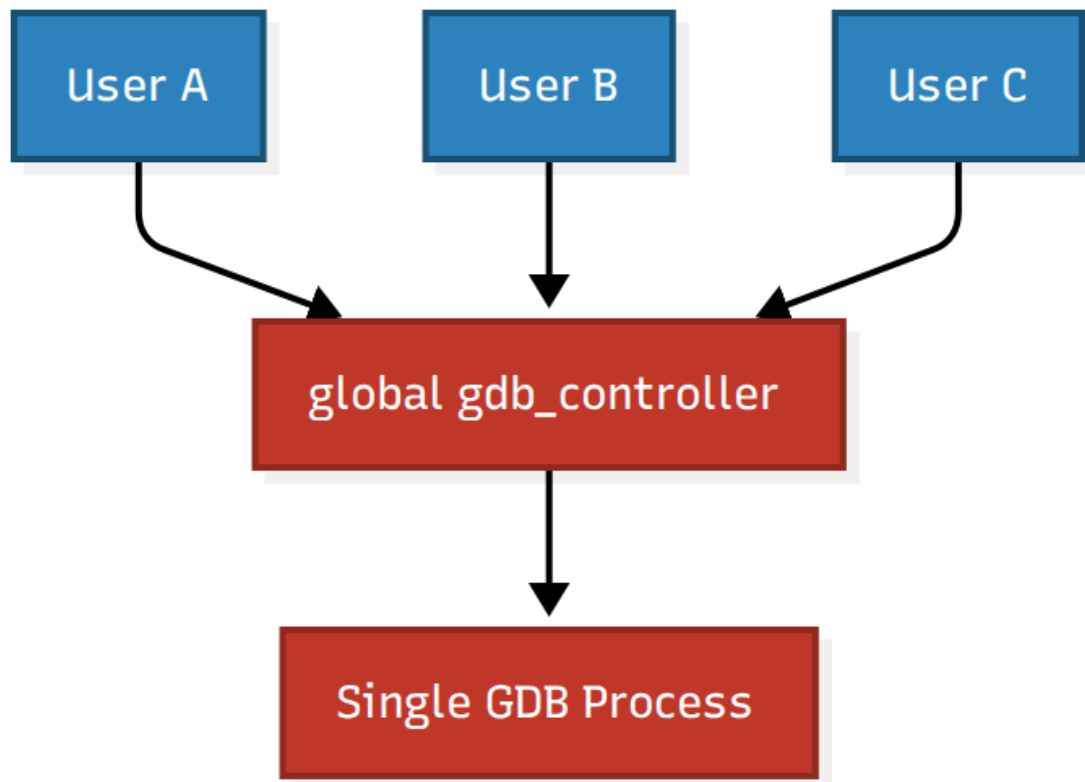


Figure 1. Current broken architecture - global gdb_controller shared across all users

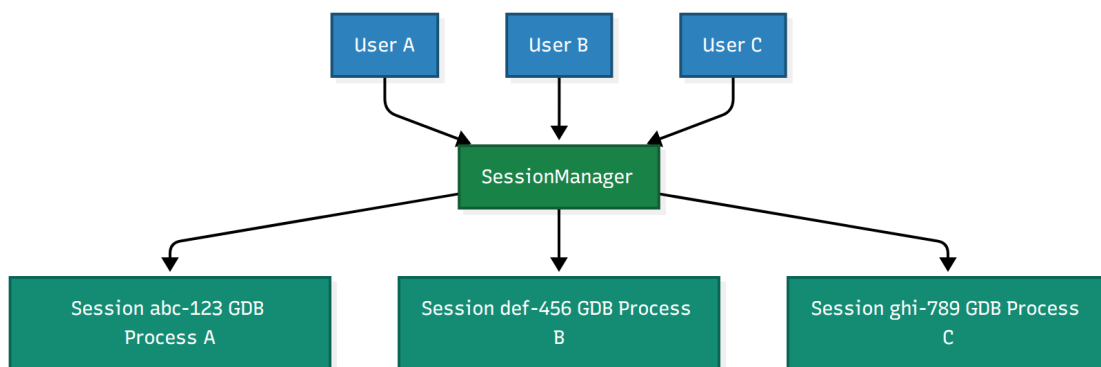


Figure 2. Proposed per-session architecture - each user routes to an isolated GDB instance

3. Proposed Solution

PHASE 1	PHASE 2	PHASE 3
Per-Session GDB Isolation	WebSocket Real-Time Streaming	Docker Sandbox Per Project
Weeks 1-4 · Already prototyped in PR #131	Weeks 5-8	Weeks 9-13

3.1 Phase 1: Per-Session GDB Isolation

Replace the global `gdb_controller` with a `SessionManager` class. Each user calls `POST /create_session` to receive a `session_id` and includes it in every subsequent request. The backend routes each request to that user's isolated GDB instance. All 19 existing Flask routes require only the addition of `session_id` extraction - no structural changes.

The `SessionManager` interface is intentionally backend-agnostic. Replacing the in-memory dictionary with Redis, or a GDB process with a Docker container, requires changes only to `SessionManager` internals. The Flask routes never need to change.

3.2 Phase 2: Real-Time WebSocket Streaming

Current flow: the frontend sends `POST /gdb_command` and waits for a complete response. The UI freezes during long operations.

New flow(Not Implemented): the frontend opens a persistent WebSocket connection. The backend streams GDB output tokens as they are produced. The terminal component renders them in real time using a React `useEffect` subscription with proper cleanup.

The hard architectural problem is **pygdbmi's blocking I/O model**. The `controller.write()` call blocks the calling thread until GDB responds. This means a naive flask-socketio integration - where the WebSocket handler calls `controller.write()` directly and tries to stream results - will block the server thread for the duration of each GDB command, preventing any other WebSocket events from being processed.

The correct architecture requires a dedicated reader thread per session. The WebSocket handler writes the command and immediately returns. A background reader thread per session loops over the GDB process stdout, parsing tokens as they arrive and emitting them to the socket. This reader thread runs for the lifetime of the session and is stopped during session cleanup.

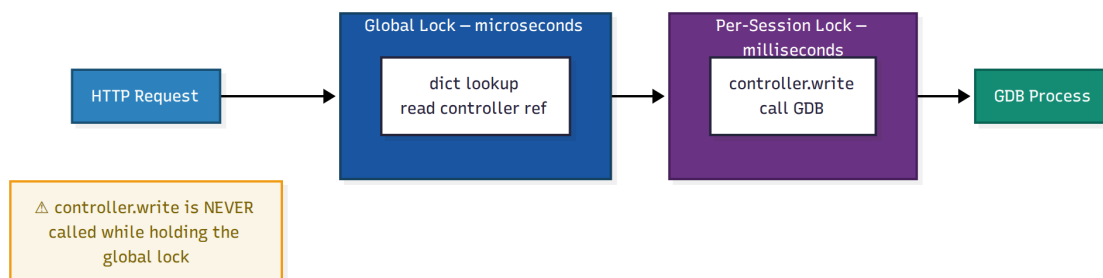


Figure 3. WebSocket streaming with per-session reader threads - the reader loop runs independently of the HTTP/WS handler

flask-socketio provides WebSocket support without requiring a full framework migration. The reason for staying with Flask rather than migrating to FastAPI is concrete: pygdbmi uses `subprocess.Popen` which is inherently blocking. Even with FastAPI's `async`, GDB calls would require `asyncio.run_in_executor` wrappers - the same threading model, more complex. Flask with per-session threading maps more naturally to how pygdbmi works.

3.3 Phase 3: Docker Sandbox Per Project

Each project gets an isolated Docker container with its own GDB instance, filesystem, and execution context. This solves the file descriptor exhaustion problem at the OS level: each container has its own fd table, completely isolated from the host.

The non-trivial parts of this phase are:

- **Container startup latency:** a new container takes 1-3 seconds to start. The session creation flow must start the container asynchronously and indicate readiness before the user can send GDB commands.
- **GDB process attachment inside a container:** pygdbmi normally launches GDB as a subprocess. Inside a Docker container, the GDB binary must be present in the container image (gdbui-sandbox). Communication happens via `docker exec <container_id> gdb --interpreter=mi`, piping stdin/stdout through the Docker API rather than directly via `subprocess.Popen`.
- **stdin/stdout piping:** the `DockerSession` class wraps the `docker exec` stream, presenting the same interface as pygdbmi's `GdbController` so `SessionManager` requires no changes.
- **Cleanup on crash:** containers that are not explicitly stopped (e.g. browser tab closes, network drop) must be cleaned up by the TTL thread. The cleanup path calls `docker stop <container_id>` in addition to the normal GDB process termination.
- **Resource limits:** containers are started with `--memory=256m` and `--cpus=0.5` to prevent any single user from exhausting host resources.

The critical design point: `SessionManager.start_gdb()` is the only method that changes between phases. Flask routes require zero changes. The React frontend requires zero changes.

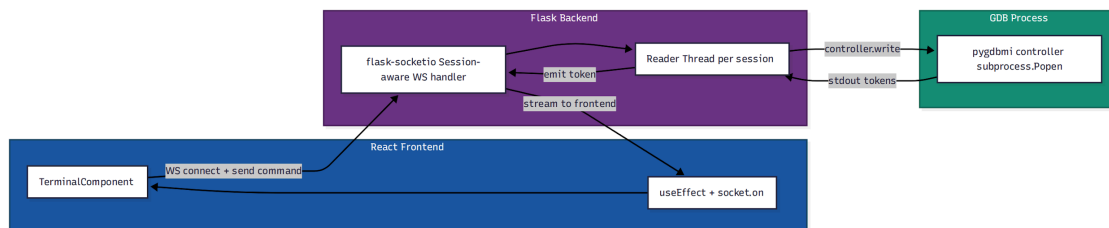


Figure 4. Docker sandbox per project - `DockerSession` implements the same interface as the in-memory session

4. Implementation Details

4.1 SessionManager Design - Two-Tier Locking

The SessionManager uses a two-tier locking strategy. A global lock protects dictionary mutations only, held for microseconds. Per-session locks serialize GDB I/O, which can take milliseconds to seconds. The critical invariant: `controller.write()` is never called while holding the global lock. This eliminates the deadlock risk entirely.

If `controller.write()` does not return within `GDB_TIMEOUT` (30 seconds), `pygdbmi` raises `GdbTimeoutError`. The per-session lock is released by normal context manager exit and the error propagates to the caller. The session stays active until the TTL thread removes it after `session_ttl` seconds of inactivity. Partial `pygdbmi` output (malformed MI tokens returned when GDB is in a bad state) is handled by `_parse_response()`, which wraps the `pygdbmi` parser in a `try/except` and returns an error payload without crashing the session.

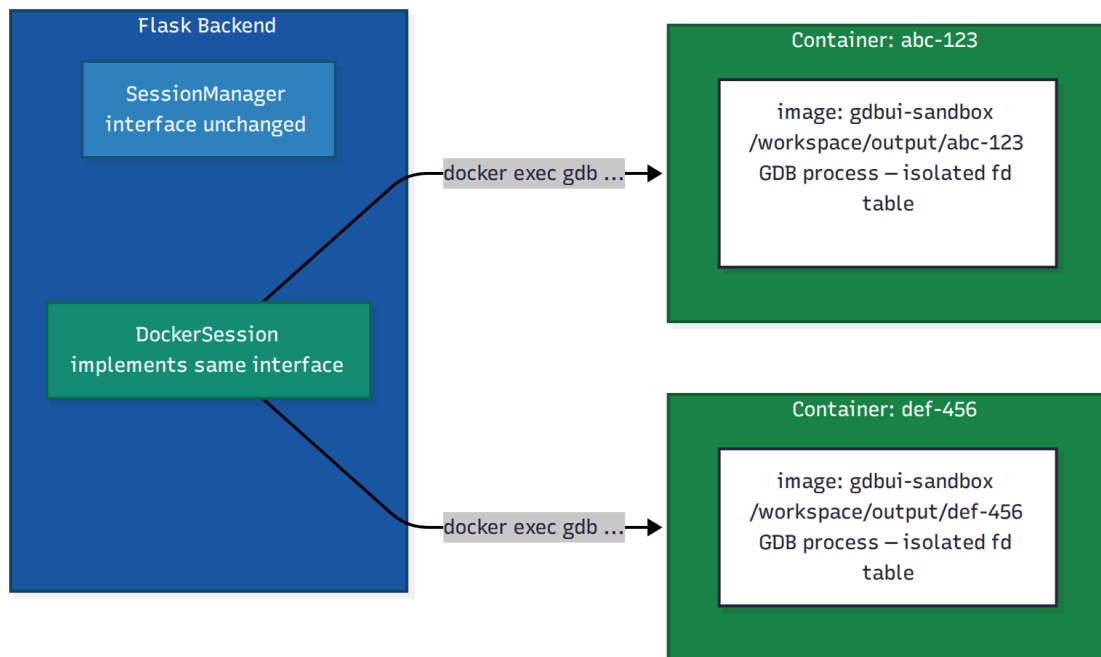


Figure 5. Two-tier locking - `controller.write()` is NEVER called under the global lock

```
class SessionManager:
    def __init__(self, session_ttl=DEFAULT_TTL):
        self.sessions = {} # dict: session_id -> state
        self.session_locks = {} # dict: session_id -> Lock
        self.lock = threading.Lock()
        self._start_cleanup_thread()

    def execute(self, session_id, command):
        session_lock = self._get_session_lock(session_id)
        with session_lock:
            # Read controller reference under global lock (microseconds)
            with self.lock:
                session = self.sessions.get(session_id)
                controller = session["controller"]
            # Call controller.write() OUTSIDE global lock (milliseconds)
```

```
response = controller.write(command)
return self._parse_response(response)
```

4.2 Cleanup Thread Race Condition

The TTL cleanup thread runs every 60 seconds and terminates sessions inactive for longer than `session_ttl`. A race condition exists: cleanup could run while a request is mid-execution on that session. This is addressed by the per-session lock: the cleanup thread acquires the per-session lock before marking a session for termination. If a request is in progress, it holds the per-session lock, and cleanup will block until the request completes. The session is never terminated mid-execution.

```
def _cleanup_expired(self):
    expired = []
    with self.lock:
        for sid, s in self.sessions.items():
            if time.time() - s["last_active"] > self.session_ttl:
                expired.append(sid)
    for sid in expired:
        # Acquire per-session lock before cleanup - blocks if request
in-flight
        lock = self._get_session_lock(sid)
        with lock:
            self.terminate_session(sid)
```

4.3 Session Lifecycle

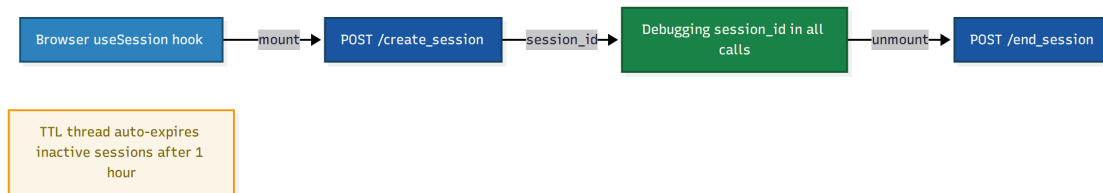


Figure 6. Session lifecycle from browser mount to cleanup

4.4 Phase 2: WebSocket Reader Thread Implementation

The reader thread per session is the architectural centerpiece of Phase 2. It is started when the WebSocket connection is established for a session and stopped when the connection closes or the session expires.

[illegible]

```
t = threading.Thread(target=reader_loop, daemon=True)
t.start()
self.sessions[session_id]["reader_thread"] = t
```

The WebSocket command handler writes the command and returns immediately - the reader thread handles all streaming output. This means the server thread is never blocked waiting for GDB, and concurrent sessions each run their own reader thread with no shared state.

4.5 Phase 3: DockerSession - GDB Inside a Container

The `DockerSession` class implements the same interface as `GdbController`. The key difference: instead of launching GDB via `subprocess.Popen`, it launches a Docker container and opens an interactive GDB process inside it via the Docker SDK's low-level `exec_create + exec_start` socket API. Note: `exec_run(stream=True)` returns a read-only generator and cannot be written to - interactive stdin requires `exec_start(socket=True)` which returns a raw bidirectional socket.

```
class DockerSession:
    """Implements the same interface as pygdbmi.GdbController."""

    def __init__(self, session_id, image="gdbui-sandbox"):
        self.client = docker.from_env()
        self.container = self.client.containers.run(
            image,
            command="sleep infinity",  # container stays alive
            detach=True,
            mem_limit="256m",
            nano_cpus=500_000_000,      # 0.5 CPU
            volumes={
                f"/workspace/output/{session_id}": {
                    "bind": "/workspace/output", "mode": "rw"
                }
            },
            network_disabled=True,      # no outbound network
        )
        # exec_create + exec_start(socket=True) gives a bidirectional
        # raw socket. exec_run(stream=True) is read-only and cannot
        # accept stdin - do not use it for interactive GDB sessions.
        exec_id = self.client.api.exec_create(
            self.container.id,
            ["gdb", "--interpreter=mi", "--quiet"],
            stdin=True, stdout=True, stderr=True,
        )
        self._sock = self.client.api.exec_start(
            exec_id, detach=False, socket=True
        )
        self._parser = GdbMiParser()  # wraps pygdbmi MI token parser

    def write(self, command):
        """Send a MI command to GDB and return parsed response tokens."""
        self._sock.sendall((command + "\n").encode())
        return self._read_response()

    def _read_response(self, timeout=30):
        """Read from socket until GDB emits a (gdb) prompt token."""
        self._sock.settimeout(timeout)
        buf = b""
        while True:
            chunk = self._sock.recv(4096)
```

```

        if not chunk:
            break
        buf += chunk
        if b"(gdb)" in buf:
            break
    return self._parser.parse_response(buf.decode(errors="replace"))

def exit(self):
    """Stop the container on session cleanup."""
    try:
        self._sock.close()
    except Exception:
        pass
    self.container.stop(timeout=5)
    self.container.remove()

```

4.6 Filesystem Isolation

Each session writes compiled binaries to `output/{session_id}/` instead of the shared `output/` directory. The directory is created when the session starts and removed on cleanup. Two users compiling files with the same name no longer interfere at the filesystem level.

Before (Collision)	After (Isolated)
output/program.exe (User A)	output/abc-123/program.exe
output/program.exe (User B overwrites)	output/def-456/program.exe

4.7 Frontend Session Integration

A `useSession` hook manages the full lifecycle: calls `POST /create_session` on mount, stores the `session_id` in React state (not `localStorage`), and calls `POST /end_session` on unmount via a `useRef`-based cleanup. This pattern correctly handles React StrictMode double-mount.

The `sessionId` is provided globally via `DataContext` so all six components (`TerminalComp`, `Breakpoint`, `BreakPoints`, `MemoryMap`, `Functions`, `Stack`) can access it without prop drilling.

A centralized `makeRequest(endpoint, data, sessionId)` helper in `api.js` automatically appends `session_id` to every request body. Adding future components requires no knowledge of the session system beyond importing the helper.

WebSocket reconnection: if the connection drops mid-debug session, the React client attempts reconnection with the existing `session_id` as a query parameter. The backend keeps the GDB process alive for the session TTL (default: 1 hour). While disconnected, the per-session reader thread continues running and buffers any spontaneous GDB output (e.g. breakpoint hits). If the session is still active when the client reconnects, debugging resumes from the current GDB state - breakpoints, call stack, and variable watches are preserved because they live in the GDB process, not the WebSocket connection. Session state is lost only if the TTL expires before reconnection, in which case the client receives a clear error and prompts the user to restart the session.

5. Session Capacity and Resource Limits

A question any mentor evaluating this proposal should ask: how many concurrent GDB sessions can a typical server handle, and what justifies the MAX_SESSIONS cap?

Each GDB session consumes approximately:

- ~15-25 MB RSS for the GDB process itself (gdb binary + loaded symbols)
- ~2-5 MB for the Python process overhead (Flask thread + pygdbmi state)
- 1 file descriptor pair for the subprocess pipe
- 1 thread for the session lock + 1 reader thread in Phase 2

On a 2GB RAM server, this gives a practical ceiling of roughly 40-60 concurrent sessions before memory pressure causes GDB startup failures. The default MAX_SESSIONS=50 is set conservatively at the low end of this range, configurable via environment variable so operators can tune for their hardware. The limit is enforced at session creation - the 51st request gets a clear HTTP 503 with a human-readable message rather than silent OOM failure.

Resource	Per Session	50-Session Total
RAM	~20 MB	~1 GB
Threads	2 (Phase 2+)	100
File descriptors	2 (pipe pair)	100
Docker containers (Phase 3)	1	50

6. Security

The original proposal's BLOCKED_COMMANDS approach is insufficient. GDB's MI interface accepts arbitrary expressions, and a simple blocklist cannot cover the attack surface. **call system("rm -rf /")** executes through GDB's expression evaluator, not as a shell command - it would not match a shell-level blocklist. Reading `/proc/self/mem` via GDB's memory inspection commands is similarly outside any command-level filter.

The correct defense has three layers:

Layer 1 - Input Sanitization (Path Traversal Only)

File path inputs (compile, upload) are sanitized against path traversal using `os.path.realpath()` and checked against the session's working directory. This is the only input type where sanitization is both necessary and effective.

Layer 2 - GDB Process Isolation

GDB command injection is addressed at the process level, not the command level. In Phase 1, GDB runs as a dedicated subprocess with a reduced umask. In Phase 3, GDB runs inside a Docker container with `network_disabled=True`, a read-only filesystem except for the session workspace, and a non-root user. Any malicious expression that executes a system call can only affect the container, which is destroyed at session end.

Layer 3 - Session-Level Limits

Each session is subject to GDB_TIMEOUT (default: 30s) for any single command. Commands that hang (e.g. attempting to call a system function that blocks) time out and mark the session for cleanup. Combined with MAX_SESSIONS, this prevents both resource exhaustion and denial-of-service from long-running expressions.

One acknowledged limitation: `session_id` is a UUID generated at session creation and used as the sole authorization token for that session. Any client that obtains a valid `session_id` can send commands to that session. This is acceptable for GDB-UI's current deployment model (local or trusted-network use) but would require an additional auth layer (signed session tokens, user accounts, or at minimum HTTPS + HttpOnly cookie binding) before public internet exposure. This is explicitly out of scope for this proposal and documented as a known limitation in the handover notes.

Vulnerability	Before	After
Path traversal	../../etc/passwd accepted	HTTP 400, blocked before GDB
GDB expression injection	call system(...) executes	Phase 3: container isolation, process destroyed at session end
Malformed JSON	Server 500 crash	HTTP 400 with error message
Unbounded sessions	Unlimited GDB processes, OOM	MAX_SESSIONS cap, HTTP 503
Filesystem collision	Users overwrite each other's binaries	Isolated to output/{session_id}/
Hanging GDB command	Thread blocked indefinitely	GDB_TIMEOUT, session marked for cleanup

Command injection checks use `validate_command()`, a module-level function called from `execute()`. Path traversal checks use `sanitize_program_name()`, called from `start_gdb()` and also directly from the `/compile` and `/upload_file` routes in `main.py`.

7. API Endpoints

Endpoint	Method	Request Body	Response
<code>/create_session</code>	POST	<code>{}</code>	<code>{ success, session_id }</code>
<code>/end_session</code>	POST	<code>{ session_id }</code>	<code>{ success }</code>
<code>/gdb_command</code>	POST	<code>{ session_id, command, name }</code>	<code>{ success, result }</code>
<code>/compile</code>	POST	<code>{ session_id, code, name }</code>	<code>{ success, output }</code>

/upload_file	POST	session_id (form), file	{ success, file_path }
/set_breakpoint	POST	{ session_id, location, name }	{ success, result }
/info_breakpoints	POST	{ session_id, name }	{ success, result }
/stack_trace	POST	{ session_id, name }	{ success, result }
/threads	POST	{ session_id, name }	{ success, result }
/memory_map	POST	{ session_id, name }	{ success, result }
/ws/debug	WS	session_id as query param	streaming GDB output tokens

Note: the `name` field in request bodies is the program filename (e.g. `hello.cpp`). It is used to locate the compiled binary in `output/{session_id}/{name}` and to name the GDB inferior. It is a legacy field from the original single-session API, retained to avoid breaking existing frontend calls during Phase 1 migration.

All routes return HTTP 400 if `session_id` is missing, HTTP 404 if not found, and HTTP 400 for malformed JSON instead of crashing with 500.

During the Phase 2 rollout, existing HTTP-based integrations continue to function unchanged: the `POST /gdb_command` route remains live alongside `/ws/debug`. flask-socketio's long-polling transport handles environments where WebSocket connections are blocked by a proxy, but this requires no additional implementation - it is built into the library. Operators can disable it via `async_mode='gevent'` if long-polling overhead is a concern.

8. Upgrade Path

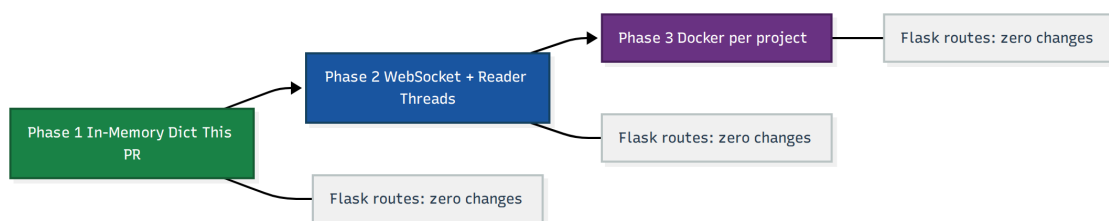


Figure 7. Backend-agnostic upgrade path - Flask routes require zero changes at every phase

This upgrade path is what makes the three-phase structure viable. Phase 1 ships a working, isolated session architecture. Phase 2 adds streaming on top of Phase 1 without touching any session management code. Phase 3 swaps only the GDB backend - the SessionManager interface, all Flask routes, and the React frontend remain unchanged across all three phases.

9. Test Coverage

15 tests were written before PR #131 was submitted, covering:

Test	What it proves
test_two_sessions_independent	Two sessions run independently with no interference
test_session_expiry	TTL cleanup auto-expires inactive sessions
test_concurrent_requests	10 simultaneous threads against one session, no crashes or corruption
test_end_session_cleanup	end_session terminates GDB process and releases resources
test_path_traversal_blocked	../etc/passwd raises ValueError before reaching GDB
test_session_filesystem_boundary	GDB working directory confirmed scoped to output/{session_id}/, cannot write outside it
test_max_sessions_limit	101st create_session raises RuntimeError
test_invalid_ttl	Negative TTL raises at construction time
test_ensure_program_switches	Switching programs within a session handled correctly
test_session_directory_created	output/{session_id}/ created on session start
test_output_directory_isolated	Two sessions write to separate directories
test_invalid_session_id	Non-existent session_id returns HTTP 404
test_missing_session_id	Missing session_id returns HTTP 400
test_session_touch_on_activity	last_active timestamp updated on each request
test_shutdown_cleans_all	shutdown() terminates all active GDB processes

Phase 2 and Phase 3 tests will be written in the same style before the relevant PRs are opened: WebSocket reader thread tests (verify tokens arrive without blocking the event loop), reconnection tests (session state survives a connection drop), and Docker container tests (container created on session start, destroyed on session end, resource limits enforced).

10. Demo

Live demo video: <https://www.loom.com/share/af7d9ecfb5924b05b044b739811cabe7>

Navigate to `/demo` after `npm run dev`. Two side-by-side panels each create independent sessions. Commands sent to Panel A do not affect Panel B. Each panel displays its own `session_id`. The Reset Session button ends and recreates the session independently. MOCK mode works without any backend running. LIVE mode connects to the real Flask server.

11. Timeline

The timeline reflects the actual difficulty distribution of the three phases. Phase 1 is already prototyped; Weeks 1-4 are integration, testing, and review. Phase 2 (WebSocket with correct reader thread architecture) is the most technically subtle and gets 4 weeks. Phase 3 (Docker with real GDB-in-container attachment) is the most operationally complex and gets 5 weeks - not 2 as in a typical optimistic timeline.

Community Bonding - May 1 to May 26

Align with mentor on session store approach for Phase 2 (in-memory vs Redis) and finalize Docker base image selection based on GDB binary availability and image size. Review all open PRs and resolve merge conflicts arising from Phase 1. Finalize edge cases identified during mentor review of PR #131. Prototype the reader thread approach locally to validate it works with the real Flask-SocketIO event model before Week 5 begins.

Week 1-2 - May 27 to June 7

Incorporate all mentor feedback on SessionManager. Full integration test with real GDB binary on Linux, including the TOCTOU race in `ensure_program` and partial `pygdbmi` output edge cases found during local testing. Update README and contributor documentation, including the deployment note that Flask must be run with

Note: the multi-session architecture requires Flask to run with `threaded=True` (the default in Flask 1.0+) or under a WSGI server such as gunicorn with `--workers=1 --threads=N`. The single-threaded development server will serialize all requests, defeating the per-session isolation. This requirement is documented in the updated README and enforced by a startup assertion in `session_manager.py`.

Week 3-4 - June 8 to June 21

Complete React frontend session lifecycle. All six components passing `session_id` consistently. Integration testing end to end. Demo page polished with error states, reconnection handling, and loading indicators.

Milestone

Phase 1 fully merged and stable. All 15 tests passing against a real GDB binary. Demo page deployed.

Week 5-6 - June 22 to July 5

Implement flask-socketio WebSocket backend. Session-aware connection management with room-based isolation. Per-session reader thread started on WebSocket connect, stopped on disconnect or session expiry. Handle the `GdbTimeoutError` loop correctly to avoid busy-waiting.

Week 7-8 - July 6 to July 19

React WebSocket client integration in TerminalComp. Real-time token rendering with useEffect socket subscription and proper cleanup on unmount. HTTP polling fallback for environments without WebSocket support. Midterm evaluation. Documentation update.

Milestone

Phase 2 fully merged. WebSocket streaming working for all existing GDB commands. Reader thread confirmed not blocking the Flask-SocketIO event loop under concurrent load.

Week 9-10 - July 20 to August 2

Write the gdbui-sandbox Dockerfile (base: ubuntu:22.04 with gdb, gcc, and a non-root user) and add it to the GDB-UI repository under docker/. Build and publish the image to GitHub Container Registry (ghcr.io/c2siorg/gdbui-sandbox) via a GitHub Actions workflow so any deployment can pull it without a local build step. Implement DockerSession class with the exec_create + exec_start(socket=True) GDB attachment approach. Container lifecycle: async create on session start, stop and remove on end_session, TTL cleanup via docker stop. Validate stdin/stdout piping through the Docker SDK produces correctly parsed pygdbmi MI output.

Week 11-12 - August 3 to August 16

Docker integration testing with concurrent sessions. Per-project filesystem isolation inside containers confirmed via integration tests. Reconnection testing: container survives a WebSocket drop and resumes correctly. Performance benchmarking under concurrent load: latency, memory footprint, container startup time measured and documented.

Milestone

Phase 3 functional with Docker. DockerSession passes equivalent test suite to in-memory session. Resource limits enforced and verified.

Week 13 - August 17 to August 25

Security audit of container escape vectors: verify network_disabled, read-only filesystem, non-root user. Final documentation. Bug fixes from integration testing. Final evaluation preparation. Handover notes for future contributors covering open questions (Redis session store, horizontal scaling, container orchestration).

12. Pre-Proposal Contributions

GDB-UI - github.com/c2siorg/GDB-UI

PR #131 (open - awaiting maintainer review)

feat: replace global gdb_controller with per-session SessionManager

New session_manager.py with two-tier locking, per-session filesystem isolation, TTL cleanup, and security fixes. 15 passing tests. A /demo page showing two users debugging independently in MOCK and LIVE modes. Submitted pre-proposal to validate the architecture before proposing it. This is the core Phase 1 deliverable.

PR #118 (open - code review feedback addressed, awaiting merge)

feat: connect Threads component to live /threads API

Replaced hardcoded mock data with real backend calls. Added AbortController cleanup on unmount and typeof guard on API response after code review feedback - implemented within the hour of review. All reviewer comments resolved; no outstanding issues.

PR #96 (open - awaiting merge, no open feedback)

fix: migrate ReactDOM.render to createRoot for React 18 compatibility

Addressed React 18 deprecation warning. Updated import from react-dom to react-dom/client.

PR #95 (open - awaiting maintainer review)

feat: add Home landing page and Login page with routing

Fixed blank screen at root route and non-functional Login button. Added hero section and feature cards describing GDB-UI's capabilities.

CircuitVerse - github.com/c2siorg/CircuitVerse

PR #7109 (open - awaiting maintainer review, no feedback yet)

fix: normalize blank mentor param before params.expect in Rails 8

Fixed a Rails 8 regression in the classroom workflow where blank mentor parameters caused a ParameterMissing exception. Identified through code exploration without an existing issue. No review comments received yet.

PR #6638 (open - awaiting maintainer review, no feedback yet)

fix: add model-level uniqueness validation to PendingInvitation

Uniqueness was previously enforced only at the database layer. Added model-level validation to surface errors through the Rails error API consistently. No review comments received yet.

13. About Me

13.1 Background

I am a 2nd year B.Tech Computer Science student at MIT Art, Design and Technology University, Pune. In my 3rd semester I interned at Locobuzz, a production SaaS platform used by enterprise clients, working as a React developer on a MERN stack with Docker and ClickUp integration, shipping 2 features to real users. Production code at a company that charges clients by uptime taught me one thing: you understand what you ship, or you don't ship it.

I lead the Google Gemini AI Club at my college with 50 active members. The logistics of running it - deciding what to build, keeping contributors engaged across a semester - taught me more about project management than any course.

In hackathons I compete seriously: top 25 of 800 in the SIH internal hackathon, TuteDude hackathon competitor, Udaan 3.0 winner at my college. Each one was a test of scoping under time pressure.

I have solved 80+ LeetCode problems and completed Operating Systems, Data Structures and Algorithms, Database Management Systems, Software Engineering, and Systems Programming courses. The concurrency concepts from OS - lock ordering, deadlock conditions, thread lifecycle - apply directly to every architectural decision in this proposal.

13.2 Projects

Helios - CPU-Only Adaptive Inference Serving Platform

Written in Go with a React observability dashboard. The core is a feedback controller that evaluates P95 latency, CPU pressure, queue depth, and memory every two seconds and dynamically adjusts worker concurrency, batch size, and admission control. I built it CPU-only intentionally to understand how scheduling decisions affect tail latency without GPU acceleration hiding the inefficiencies.

The key finding: increasing concurrency past core count worsens tail latency due to context-switching overhead. That finding directly informed the SessionManager design. Serializing all users under a single global lock would recreate the same contention I measured in Helios. The two-tier locking - global lock for dict mutations only, per-session lock for GDB I/O - exists because of what Helios showed me, not because it appeared in a textbook.

GitHub: github.com/Uday9909/Helios

Sentinel AI - Real-Time Incident Command Platform

Built on Go, Apache Kafka, Python with IsolationForest for anomaly detection, Ollama with Llama 3.2 for root cause analysis, and a React + Vite dashboard with Elasticsearch integration. Fully containerized with Docker Compose. Instead of showing raw error logs it explains why the system broke. Building it gave me hands-on experience with streaming pipelines, feedback-driven observability, and Docker-based isolation - all directly relevant to Phase 2 and Phase 3 of this proposal.

GitHub: github.com/Uday9909/Sentinel-Ai

13.3 Technical Skills

Area	Technologies
------	--------------

Frontend	React, Context API, WebSockets, Axios, React Router, Vite
Backend	Flask, Python, Go, Node.js, REST APIs, pygdbmi, subprocess management
Streaming	Apache Kafka, Elasticsearch, WebSockets, flask-socketio
DevOps	Docker, Docker Compose, GitHub Actions, CI/CD
Databases	MongoDB, PostgreSQL
Languages	JavaScript, Python, Go, C++
Tools	GDB, Git, Postman, Figma

13.4 Why I Am Applying to C2SI

I read Shubh's 2024 journey before writing a single line of this proposal. What stood out was not the proposal itself - it was that he had a working prototype before submitting. I share that approach. PR #131 exists because I wanted to understand whether the architecture I was proposing actually worked before I proposed it. It does, with caveats I found while building it and documented in the security and implementation sections above.

The Docker sandbox work in Phase 3 is genuinely open. How GDB behaves inside a container under different base images, what startup latency looks like under real load, whether the docker exec approach produces correctly parseable MI output - I don't fully know yet. That's why it's interesting, and why the timeline allocates 5 weeks instead of 2.

13.5 Availability

I am available 35-40 hours per week throughout the GSoC period. My university exams end before May. I have no travel or obligations planned between May and August. IST overlaps comfortably with C2SI mentor hours.

13.6 After GSoC

My goal after the summer is to remain an active contributor and eventually a reviewer for new GSoC candidates working on GDB-UI. The Docker sandbox work in Phase 3 will leave open questions worth following - Redis session store, horizontal scaling across multiple Flask instances, Kubernetes-based container orchestration - and I intend to work on them past the summer.

14. Why I Am the Right Person

Before writing this proposal, I opened PR #131 implementing Phase 1 with 15 passing tests. I wanted to verify the architecture worked before proposing it.

The constraints I ran into while building it - file descriptor limits when GDB spawns child processes, pygdbmi's blocking behavior in a multithreaded Flask context, the TOCTOU race in `ensure_program` when two requests check the same session simultaneously - were not things I anticipated. I found them while building and fixed them before opening the PR. They shaped the security section and the cleanup thread design in this proposal.

The WebSocket architecture in Phase 2 has a non-obvious problem: naive flask-socketio integration will block the server thread. I know this because I tried it locally. The reader thread solution in Section 4.4 came from working through why the naive approach fails, not from reading about it. That kind of concrete finding is why the Phase 2 section of this proposal is more specific than the original - it reflects what I actually ran into rather than what I expected to run into.

When a reviewer flagged missing `AbortController` cleanup and a typeof guard in my Threads PR, I implemented both fixes within the hour. I try to act on feedback without debating it first.

I am a second-year student. There are things in Phase 3 - specifically how the Docker SDK handles GDB's MI protocol streams - that I have not fully figured out yet. The timeline reflects that honestly. My goal is to do this one well, not to oversell how smooth it will be.

GitHub: github.com/Uday9909 · Email: writetoudaybir@gmail.com · Timezone: IST (UTC+5:30)